المدرسة العليا للتكنولوجيا
École Supérieure de Technologie
ⵜⴰⵎⴰⵡⴰⵙⵜ ⵏ ⵓⵙⴳⵎⵉ ⴰⵏⴰⵎⵓⵔⵓ ⴷⴰⵔ ⵓⵏⴰⵎ

Chapitre 3
Gestion des fichiers

Programmation Python/M245

Anas MOKHTARI
a.mokhtari@ump.ac.ma

2025/2026

CDL, Département Informatique, Ecole Supérieure de Technologie - UMP, Oujda

Plan du chapitre

Les fichiers sous Python

Fonctions & Méthodes

Cas de fichiers binaires

Manipuler les fichiers CSV

Conclusion

Les fichiers sous Python

Les fichiers sous Python

- Les programmes en Python communiquent avec les fichier par des entités abstraites appelées handles ou streamers
- L'ouverture et la fermeture du fichier est l'opération de connecter le stream avec un fichier
- Mode d'ouverture : la manière par laquelle le stream sera traité
- Modes basiques :
  - Mode lecture (read mode) : opérations de lecture seulement,
  - Mode écriture (write) : opérations d'écriture seulement,
  - Mode mise à jour (update) : opérations de lecture et d'écriture.

Les fichiers sous Python

- En Python, chaque fichier est caché derrière un objet de classe adéquate
- Cet objet est créé au moment de l'ouverture du fichier, et supprimé au moment de sa fermeture
- Cet objet vient d'une des classes suivantes :
  - IOBase
    - RawIOBase
    - BufferdIOBase
    - TextIOBase
- `open()` : c'est la fonction utilisée pour ouvrir un fichier
- `close()` : c'est la méthode utilisée pour fermer un fichier
- Il y a deux types de streams :
  1. Streams texte : Lire/écrire caractère par caractère ou ligne par ligne,
  2. Streams binaires : Lire/écrire octet par octet ou bloc par bloc.

Les fichiers sous Python

- La fin d'une ligne est codée par :
  - Le caractère `LF` (`\n`) dans les systèmes Unix,
  - Les caractères `CR` et `LF` (`\r\n`) dans les systèmes Windows.
- En Python, la manipulation des fins de ligne est complètement transparente :
  - Pour un stream texte, Python traduit la fin de ligne en fonction du système au moment de l'exécution du programme
  - Dans le cas contraire, et par défaut, Python utilise la représentation d'Unix

Fonctions & Méthodes

Fonctions & Méthodes : `open()`

- Syntaxe :
  `<stream> = open(<file>, mode=<mode>, encoding=<encodage>)`
  - `<stream>` : c'est l'objet retourné par la fonction open(),
  - `<file>` : le nom du fichier à ouvrir,
  - `<mode>` : 'tr' par défaut,
  - `<encodage>` : comme UTF-8 pour les fichiers texte
- Exemple : `fichier1 = open("/etc/fstab", mode='r')`

Fonctions & Méthodes : `open()`

- Modes possibles :
  - `r` : read only,
  - `w` : write only. Si le fichier existe, il sera écrasé,
  - `a` : append mode. Si le fichier existe, les nouvelles données seront ajoutées à la fin du contenu de ce fichier
  - `r+` : read and update (write) mode. Le fichier doit exister !
  - `w+` : write and update mode. L'existence du fichier n'est pas exigée
- Modes texte/binaure :
  - `b` : pour le mode binaire,
  - `t` : pour le mode texte.

Fonctions & Méthodes : `open()`

- Si le fichier est introuvable, Python déclenche une exception `FileNotFoundError`
- Exemple 1 :

```
# Le mode d'ouverture par défaut est 'r'
f = open("test.txt")
f = open("C:/Python33/README.txt")
```

Fonctions & Méthodes : open()

- Exemple 2 :

```
>>> f = open("test.txt", 'r', encoding = 'utf-8')
>>> f.read(4) # lire les 4 premières données
'Ceci'
>>> f.read(4) # lire les 4 prochaines données
' est'
>>> f.read() # lire dans le reste jusqu'à la fin du fichier
' mon premier fichier\nCe fichier\ncontient trois lignes'
>>> f.read() # une lecture supplémentaire renvoie une chaîne vide
''
```

Fonctions & Méthodes : `.close()`

- Syntaxe : `<nom_objet_stream>.close()`

Fonctions & Méthodes : `.read()`

- Syntaxe : `<objet stream>.read([n])`
  - Si `n` spécifiée : retourne les `n` caractères lus depuis le fichier,
  - Si `n` n'est pas spécifiée, tout le contenu du fichier est retourné,
  - S'il n'y a plus de caractère à lire, une chaîne de caractères vide est retournée.

Fonctions & Méthodes : `.readline()`/`.readlines()`

- `<objet stream>.readline()` : lit toute une ligne de texte depuis le fichier
- `<objet stream>.readlines([n])` :
  - Lit le contenu du fichier, sans dépasser n caractères, et retourne une liste de chaînes de caractères, un élément par ligne du fichier,
  - S'il n'y a plus rien à lire, retourner une ligne vide.
- Exemple 1 :

```
>>> f.readline()
[]
>>> f.seek(0) # amener le curseur de fichier à la position initiale
0
>>> f.readlines()
['Ceci est mon premier fichier\n', 'Ce fichier\n', 'contient trois
↪  lignes']
```

Fonctions & Méthodes : `.readline()`/`.readlines()`

- Exemple 2 :

```
>>> f.readline()
'Ceci est mon premier fichier\n'
>>> f.readline()
'Ce fichier\n'
>>> f.readline()
'contient trois lignes'
>>> f.readline()
''
```

Fonctions & Méthodes : .readline()/.readlines()

- Exemple 3 : en utilisant la boucle for

```
contient trois lignes
>>> f.seek(0) # amener le curseur de fichier à la position initiale
0
>>> for line in f:
...     print(line)
...
...
Ceci est mon premier fichier

Ce fichier

contient trois lignes
```

Fonctions & Méthodes : `.tell()`/`.seek()`

- `tell()` : retourne la position actuelle du curseur dans le fichier
- `seek(n)` : remet la position du curseur à la position `n`
- Exemple :

```
>>> f.tell()
63
>>> # obtenir la position actuelle du fichier
>>> f.seek(0) # amener le curseur de fichier à la position initiale
0
>>> print(f.read())
Ceci est mon premier fichier
Ce fichier
contient trois lignes
```

Fonctions & Méthodes : `.write()`

- `<objet stream>.write(<chaîne de caractères>)` : écrit la chaîne de caractères dans le fichier
- Exemple :

```
>>> fichier = open("fichier.txt","a")
>>> fichier.write ("\n une autre ligne ajoutée à la fin")
>>> fichier.close ()
```

Context manager

- En utilisant le mot-clé `with`
- Pas besoin de fermer le fichier
- Exemple 1 :

```
with open("file.txt", "r") as fichier:
    print (fichier.read())
```

- Exemple 2 :

```
with open("test.txt",'w',encoding = 'utf-8') as f:
    f.write("my first file\n")
    f.write("This file\n\n")
```

Cas de fichiers binaires

Fichier binaire : Données amorphes et bytearrays

- Ce sont des données qui n'ont pas de forme spécifique
- Elles ne peuvent être stockées ni dans une chaîne de caractères ni dans des listes
- Il faut utiliser des conteneurs spéciaux pour manipuler ces données, comme la classe `bytearray`
- Déclaration : `donnees = bytearray(100)`
- Les `bytearrays` sont :
  - Modifiables,
  - Utilisables avec la fonction `len()` ,
  - Accessibles en utilisant l'indexation.
- Les éléments du `Bytearray` doivent être des entiers, compris entre 0 et 255
- Les `bytearrays` sont utilisés pour lire/écrire les fichiers binaires

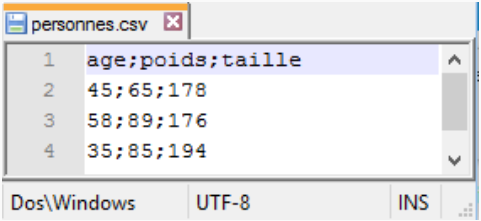
Fichier binaire : Méthodes `readinto()` et `read()`

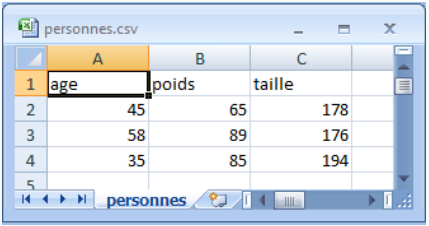
- `<objet stream>.readinto(<nom du byte array>)` : Lit le contenu du fichier binaire, le met dans le *byte array* et retourne le nombre d'octets lus avec succès
- `<objet stream>.read()` : Lit le contenu du fichier binaire et le retourne dans un objet de la classe bytes. Cette classe ressemble à celle de bytearray, mais elle est immuable
- Exemple : `donnees = bytearray(bf.read())`

Manipuler les fichiers CSV

Fichiers CSV : Description

- `csv` : comma-separated values
- Fichier texte avec structure tabulaire
- Peut être généré à partir d'un tableur (Excel, Calc, ...)
- Le format csv est utilisé pour le traitement statistique des données
- Exemple





Fichiers CSV : Module csv

- Le module csv facilite la manipulation des fichiers csv :

```
Import csv

f = open("fichier.csv", "r")
contenu = csv.reader(f, delimiter=" ;")
for ligne in contenu :
    print(ligne)
f.close()
```

```
['age', 'poids', 'taille']
['45', '65', '178']
['58', '89', '176']
['35', '85', '194']
```

- Chaque ligne est une liste

Fichiers CSV : Module csv

- Le module csv facilite la manipulation des fichiers csv :

```
Import csv

f = open("fichier.csv", "r")
contenu = csv.DictReader(f, delimiter=" ;")
for ligne in contenu :
    print(ligne)
f.close()
```

```
{'poids': '65', 'age': '45', 'taille': '178'}
{'poids': '89', 'age': '58', 'taille': '176'}
{'poids': '85', 'age': '35', 'taille': '194'}
```

- Chaque ligne est une dictionnaire

Conclusion